NEXTGEN DENTAL · AUSTIN, TX

Production Analytics: Architecture & Learning Guide

How patient-privacy-minimized analytics works, where everything lives, and how to operate it.

Last verified: **2026-08-09 (evening, US Central)** · Read-only documentation — nothing was changed to produce this guide.

Labels used throughout: VERIFIED INFERENCE ASSUMPTION UNKNOWN RECOMMENDED

1. Executive summary

> **Plain English:** Your website now measures how patients use it — which pages they read, which buttons they click, which articles lead to appointment bookings — using an analytics system that **you own end-to-end**. No Google Analytics, no advertising trackers, no cookies, no patient information. The measuring script on the website only sends five pre-approved kinds of events, each stripped and validated before it leaves the browser. The data lands in your own small server in AWS, and you read it at **analytics.nextgendentalaustintx.com** behind two passwords.

The platform went live on **2026-08-09**. It consists of three cooperating parts: (1) a **tracker** on the website — a script plus a validation wrapper covered by 125 automated tests that run on every code change; (2) a **collection server** — the Umami analytics application and a PostgreSQL database running in Docker on a dedicated EC2 instance; (3) an **operating shell** — Terraform-defined infrastructure, daily backups to encrypted S3, daily disk snapshots, CloudWatch alarms, and administrative access only through AWS SSM (no SSH port exists).

Two honest caveats front and center: the **first backup and the restore drill have not yet been verified** (the stack is less than a day old — see §11 and §18), and this document distinguishes carefully between what is VERIFIED and what remains UNKNOWN.

2. Architecture at a glance

The system in one table

LAYER	WHAT RUNS THERE	WHERE	STATUS
Website	Astro static site + analytics tracker + event wrapper	S3 + CloudFront (existing website infrastructure)	VERIFIED
Collection	Caddy (TLS + auth gate) → Umami app → PostgreSQL 16	Docker Compose on one dedicated EC2 `t4g.small`, us-east-2	VERIFIED
Storage	Postgres data files	Encrypted 20 GB EBS volume mounted at `/data`	VERIFIED
Backups	Nightly `pg_dump` → S3 (KMS, versioned) + daily EBS snapshots (7 kept)	S3 bucket `nextgendental-analytics-backups` + EBS snapshots	CONFIGURED, first run unverified
Access	Dashboard: Caddy basic-auth + Umami login. Admin shell: SSM only	Browser / AWS Console	VERIFIED
Change control	PR validation (tests block) → staging → human production approval	GitHub Actions + protected environment	VERIFIED

3. Verified facts & unknowns — evidence matrix

Every load-bearing architecture fact, with its evidence

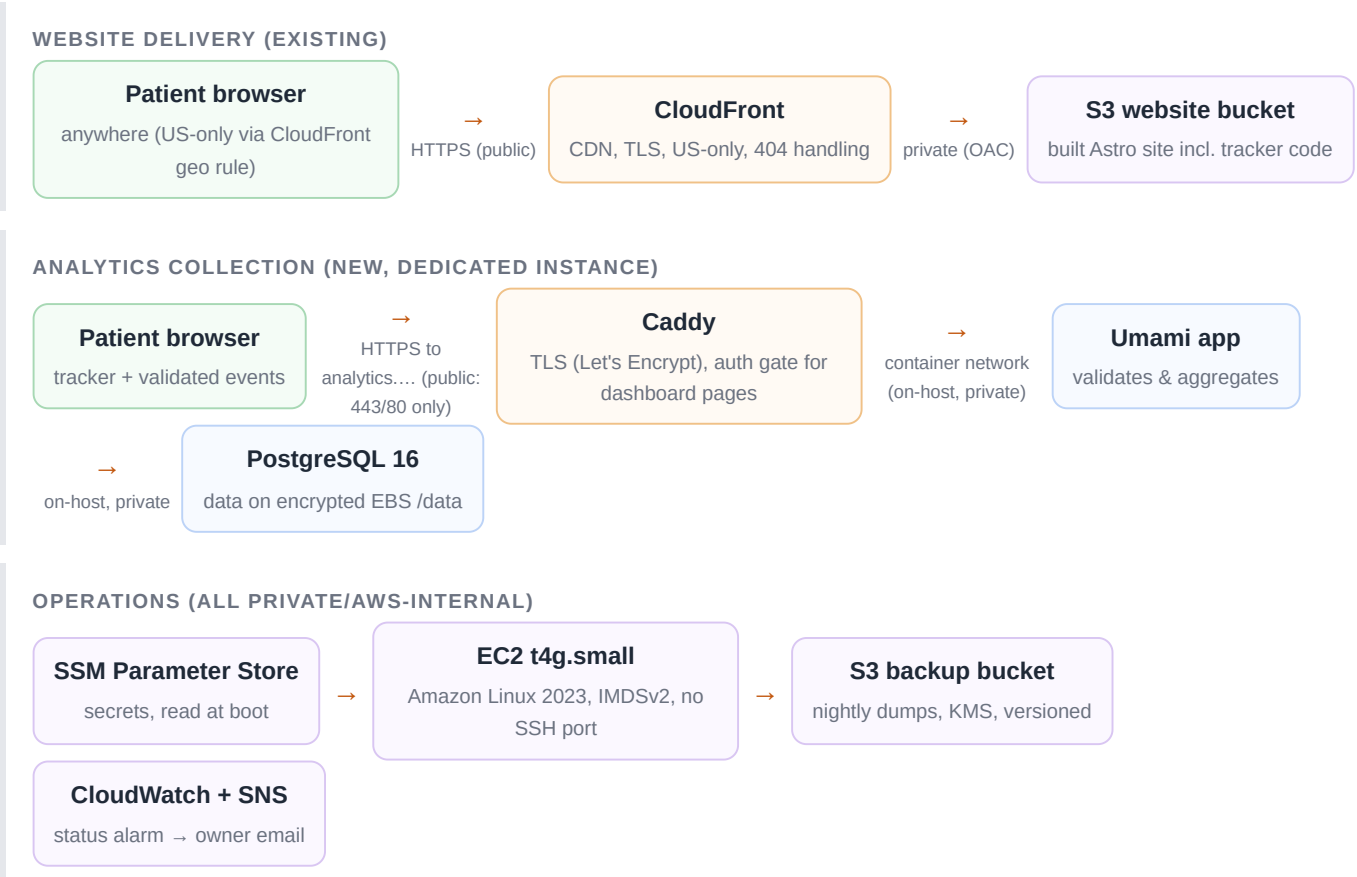
ARCHITECTURE FACT	STATUS	EVIDENCE SOURCE	LAST VERIFIED	CONFIDENCE
Analytics hostname `analytics.nextgendentalaustintx.com` → EIP of the analytics instance (Route 53 A record)	VERIFIED	Terraform apply output; live TLS certificate issuance; dashboard reachable	2026-08-09	High
Umami compute: Docker Compose on dedicated EC2 `t4g.small`, Amazon Linux 2023, us-east-2	VERIFIED	Terraform apply; live `docker ps` during SSM session (3 containers Up)	2026-08-09	High
PostgreSQL 16 container on the **same instance**; data files under `/data/pg` on a dedicated EBS volume (not RDS)	VERIFIED	docker-compose definition in bootstrap; live `docker ps`	2026-08-09	High
Encryption at rest: root EBS + data EBS + backup bucket (KMS) all encrypted	VERIFIED	Terraform plan/apply (`encrypted=true`, SSE-KMS)	2026-08-09	High
Backup schedule: nightly `pg_dump` cron → S3; daily EBS snapshot, 7 retained (DLM)	CONFIGURED	user-data cron file + DLM policy in Terraform; **first execution not yet observed**	2026-08-09	Medium
Restore test	NOT DONE	— (required before "stable"; see §11)	—	—
Session replay & heatmaps: no replay module is loaded by the site tracker; menu items exist in the Umami UI but collection requires site-side instrumentation that is deliberately absent	VERIFIED (code)	`Analytics.astro` loads only `script.js`; no replay/heatmap scripts anywhere in repo	2026-08-09	High (code) / Medium (ongoing — re-check after Umami upgrades)
Tracker endpoint `https://analytics…/api/send`; tracker script `/script.js` publicly served	VERIFIED	Caddy config; live 200 on script.js; events visible in dashboard	2026-08-09	High
Cookieless collection (no cookies, no localStorage identifiers set by analytics)	VERIFIED	Umami tracker design + owner's staging DevTools check before production approval	2026-08-09	Medium-High (re-verify after upgrades)
Query strings & fragments excluded from collected URLs	VERIFIED	`data-exclude-search/hash` attributes in `Analytics.astro`; owner's staging payload check	2026-08-09	High

Authentication: dashboard pages behind Caddy basic-auth + Umami login; API behind Umami token auth; admin shell via SSM Session Manager only (no SSH port)	VERIFIED	Live Caddyfile; security group (443/80 only); live login flow	2026-08-09	High
Website ID configuration: hardcoded per-hostname map in `Analytics.astro` and the wrapper (dev vs production), runtime switch	VERIFIED	Source on `main` (commit `fa70d23`)	2026-08-09	High
Umami application version	UNKNOWN (unpinned)	Image tag `postgresql-latest` — pin recommended (§15)	—	—
Infrastructure code merged to main	PENDING	Live stack matches branch `infra/analytics-pilot` (unmerged PR)	2026-08-09	High

4. Complete architecture diagram

Production architecture — two independent halves that meet only in the patient's browser

Left: the website you already had. Right: the new analytics stack. If the right half fails entirely, the website is unaffected.



Node types: people · public-facing · private application · AWS-internal service

If it fails: CloudFront/S3 outage → website down (analytics irrelevant). Analytics EC2 outage → website fine, tracker requests fail silently in patients' browsers, data for the outage window is lost (not queued) — recovery in §11.

5. Analytics data flow — from a patient's visit to your dashboard

> **Plain English:** when someone reads your Invisalign page and taps "Call Us," their browser sends two tiny, anonymous notes to your analytics server: "someone viewed /services/invisalign" and "someone tapped a phone link on a service page." No name, no number dialed, no cookie. Umami files those notes; the dashboard summarizes them.

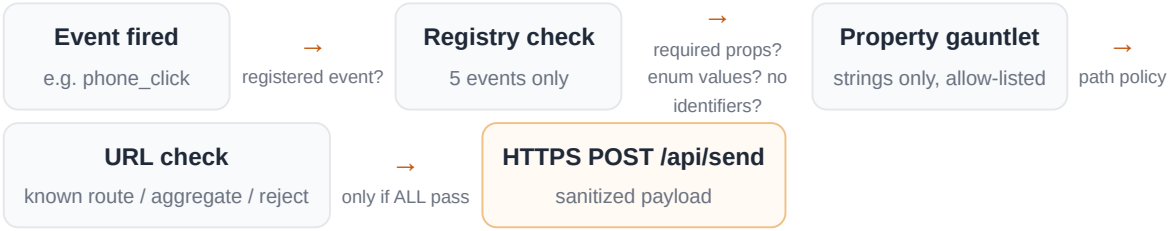
End-to-end flow (verified sequence, corrected to match production)

Steps 1–7 happen in the patient's browser; 8–10 on the analytics instance; 11–13 when you log in; 14–15 are the improvement loop.

1. Patient opens a page → CloudFront serves the static Astro page from S3. VERIFIED
2. An inline snippet checks the hostname: on `dev.`/apex/`www` it injects `script.js` from the analytics server with the matching website ID; on any other host (localhost, previews) **nothing loads at all**. VERIFIED
3. The Umami tracker records a pageview automatically — with query strings and #fragments excluded (`data-exclude-search/hash`). VERIFIED
4. User actions that match one of **five approved events** (phone click, Zocdoc click, blog→service click, form start, form success) are passed to the **event wrapper** — never raw to the tracker. VERIFIED
5. The wrapper validates: registered event? required properties present? every value on the enum allow-list? no emails/phone-numbers/free text/identifiers? ≤8 properties? Strings only? VERIFIED — 125 unit tests, blocking in CI
6. The page path is checked against a route allow-list; unknown first-party routes are aggregated to a category (never the raw path); foreign or identifier-like URLs cause the whole event to be **rejected** — only counts are kept, never the URL. VERIFIED
7. Approved payloads go by HTTPS `POST` to `/api/send`. Rejections never leave the browser. VERIFIED
8. Caddy passes `/api/*` straight to Umami (this path is public by necessity — every patient's browser must reach it). VERIFIED
9. Umami validates the website ID, assigns the event to an anonymous session (daily-rotating hash — resets every day, not traceable across days). VERIFIED (Umami design)
10. PostgreSQL stores the record on the encrypted volume. VERIFIED
11. You open the dashboard → Caddy demands basic-auth for pages → Umami demands its own login. VERIFIED
12. Umami queries PostgreSQL and aggregates. VERIFIED
13. Dashboard renders traffic, events, funnels, journeys. VERIFIED
14. The agentic monitoring process (weekly watchdog + on-demand analysis) may read **aggregate** data and produce recommendations. VERIFIED (review-only mandate)
15. Any website change resulting from a recommendation goes through PR → staging → **your explicit approval** → production. The agent cannot skip this. VERIFIED (protected environment)

Event-validation flow (inside the browser, before anything is sent)

Every custom event runs this gauntlet. Failing ANY check means nothing is transmitted; rejected values are never echoed even into error logs.



On failure: event silently dropped; an error *code* (never the value) appears in the browser console debug log; unsafe-path rejections increment a counter readable as an aggregate number.

6. Component-by-component (What / Why / Where / When / Who / How)

Every component answering the 5W1H. "Owner" = who operates/approves; Ratik holds final approval throughout.

COMPONENT	WHAT & WHY	WHERE & WHEN	WHO & HOW IT CONNECTS
Patient browser	Runs the site and the tracker. Why: analytics must observe real behavior where it happens.	Patient's device; whenever they visit.	Owned by the patient. Talks HTTPS to CloudFront (pages) and to the analytics endpoint (events).
Astro website	The static site — pages, forms, and the analytics integration are all built into its files. Why: static = fast, cheap, small attack surface.	Built by CI; served from S3/CloudFront; rebuilt on every merge to main.	Ratik approves changes. Source: GitHub `ratikk/NextGen-Dental`.
CloudFront	CDN: TLS, caching, US-only geo rule, real 404s. Why: speed + shield for the S3 origin.	AWS edge, always on.	AWS-managed. Reads S3 via Origin Access Control (private).
S3 website bucket	Holds the built site files. Why: durable, cheap static origin.	us-east-2; updated by the deploy pipeline only.	Pipeline writes via scoped OIDC role; CloudFront reads; public access blocked.
Analytics tracker (`script.js`)	Umami's standard collection script. Why: records pageviews and carries approved events; cookieless by design.	Served from the analytics instance; loads on tracked hostnames only, per page view.	Injected by `Analytics.astro`; posts to `/api/send`.
Event wrapper (`trackApprovedEvent`)	The privacy firewall: allow-listed events/properties, identifier rejection, path policy, no-echo errors. Why: makes "no PHI in analytics" a property of the code, not a promise.	Bundled into the site; runs in-browser before any custom event is sent.	125 unit tests run as a **blocking** CI step on every PR. Only it may call the tracker's event API.

DNS (Route 53)	Maps names to servers. Why: patients and the tracker find the right machines.	Hosted zone for the domain; `analytics.` A-record → the instance's static IP (EIP).	Managed via Terraform + approved changes.
TLS certificates	Website: AWS Certificate Manager (at CloudFront). Analytics: Let's Encrypt, obtained and auto-renewed by Caddy. Why: everything is HTTPS.	Renewal is automatic in both cases.	ACM auto-renews; Caddy renews ~30 days before expiry (renewal window observed in live logs).
Caddy (reverse proxy)	The analytics front door: terminates TLS, lets `/api/*` + `/script.js` through, demands basic-auth for dashboard pages. Why: one hardened entry point; no load balancer needed at this scale.	Container on the analytics instance; always on; access log (IP-truncated) on /data.	Forwards to Umami over the private on-host container network.
Umami application	The analytics brain: sessionizes events, aggregates, serves the dashboard and its API. Why: privacy-first, self-hostable, owner-controlled.	Container, always on. Version: image tag `postgresql-latest` **UNPINNED — see §15**.	Reads/writes PostgreSQL on the same host; auth via its own accounts + tokens.
PostgreSQL 16	The record system — every pageview/event row lives here. Why: Umami's supported store; SQL for future custom reporting.	Container; data files at `/data/pg` on the encrypted EBS volume; always on.	Reachable ONLY from Umami on the same host — port 5432 is not exposed to the network.
Compute host (EC2)	The one machine running all three containers. Why:	`t4g.small`, us-east-2, dedicated to analytics; auto security patches (dnf-automatic).	No SSH port. Admin = AWS SSM Session Manager (IAM-

	right-sized (~$12/mo) for a practice website's traffic.		gated, sessions logged). IMDSv2 enforced.
Storage volume (EBS)	20 GB encrypted disk holding the database + Caddy state. Why: survives instance replacement; snapshot-able.	Attached at `/data`; bootstrap formats it ONLY if blank.	Snapshotted daily by DLM (7 kept).
Secrets (SSM Parameter Store)	DB password, app secret, dashboard auth hash — encrypted SecureStrings. Why: no secrets in code, Terraform, or images.	Read once at instance boot by the bootstrap script.	Instance role may READ its own three parameters and nothing else; writing requires human AWS access.
Backups	Nightly database dump → encrypted S3 (35-day lifecycle) + daily volume snapshots. Why: two independent recovery paths.	Cron on the instance (nightly); DLM daily.	**First run pending verification** — see §11.
Monitoring	CloudWatch instance-status alarm → SNS email to Ratik; weekly agent watchdog checks tracker availability. Why: silent analytics death loses data invisibly.	Alarm continuous; watchdog weekly (Thursdays).	SNS subscription confirmed 2026-08-09.
Terraform	The infrastructure's source of truth (~25 resources). Why: reproducible, reviewable, destroyable.	`infra/analytics/`; state in the shared encrypted state bucket with locking.	Plans run automatically on PRs (read-only role); this stack's apply was CloudShell-admin by explicit approval. **Branch merge pending**.
CI/CD pipeline	Validate (tests/build/scans)	GitHub Actions, on every PR and merge.	OIDC roles per stage; protected `production`

	→ build-once artifact → staging deploy → **human gate** → production. Why: no unreviewed change can reach patients.		environment; Ratik is the sole approver.
Umami dashboard	Where you read the data (see §8).	`analytics.nextgendentalaustintx.com`, on demand.	Two auth layers; accounts managed inside Umami by you.
Agentic monitoring	Weekly watchdog + on-demand analysis; reads aggregates, writes reports and recommendations. Why: the observe–reason–learn loop.	Scheduled cloud session (Thursdays) + working sessions.	**Review-only**: cannot deploy, merge, approve, or modify anything in production.
Human approval gate	You. Every production change — website, infrastructure, rollback — stops until you explicitly approve. Why: the non-negotiable safety property of the whole system.	GitHub protected environment + "Approve …" phrases in working sessions.	Cannot be bypassed by the agent or the pipeline; vague answers are not approval.

7. Where do I go?

Safe navigation map — no credentials or sensitive identifiers included

PURPOSE	LOCATION	WHAT YOU CAN DO	ACCESS REQUIRED
View the website	https://nextgendentalaustintx.com	Verify the public site as patients see it	Public
View analytics	https://analytics.nextgendentalaustintx.com	Traffic, events, sessions, funnels, performance	Basic-auth + Umami login
Website & tracker code	github.com/ratikk/NextGen-Dental → `src/components/Analytics.astro`, `src/utils/analytics/`	Review exactly what the tracker can and cannot send	Repository access
Infrastructure code	Same repo → `infra/analytics/` (branch `infra/analytics-pilot` until merged) + `docs/ANALYTICS.md`	Review the Terraform definition of everything in §4	Repository access
The analytics server	AWS Console → EC2 → us-east-2 (Ohio) → instance named `analytics-umami`	Inspect health, CPU, status checks	Authorized AWS access
Admin shell (no SSH)	AWS Console → Systems Manager → Session Manager, or `aws ssm start-session`	Container status, logs, controlled maintenance	Authorized AWS access (sessions are logged)
Database status	Via the admin shell: `docker ps`, `docker logs` — status only, never patient-level browsing (there is none to browse)	Confirm PostgreSQL is up; check disk usage	Restricted
Backups	AWS Console → S3 → `nextgendental-analytics-backups` → `pgdump/`; EC2 → Snapshots	Confirm nightly dumps and daily snapshots exist	Restricted
DNS	AWS Console → Route 53 → nextgendentalaustintx.com zone	Inspect the `analytics` A-record	Authorized access
Certificates	Website: ACM (us-east-1). Analytics: automatic on-instance (Let's Encrypt via Caddy — check padlock in browser)	Confirm validity/expiry	Authorized / public padlock
Logs	CloudWatch Logs group `/nextgendental/analytics` (30-day retention); Caddy access log on the instance	Diagnose availability and application errors	Restricted
Deployments	GitHub → Actions tab	Every build, staging deploy, approval, and production release with full history	Repository access

8. How to use the dashboard

> **Plain English first — the three numbers people confuse:** a **View** is one page being loaded once. A **Visit** (session) is one person's whole browsing sitting — maybe five views in ten minutes. A **Visitor** is the person — but since we're cookieless, "the person" is an anonymous daily hash: the same patient returning tomorrow counts as a new visitor. That's the privacy trade-off, accepted by design: slightly inflated visitor counts in exchange for tracking nobody.

How visitor counting actually works — and why we cannot know who visited

The mechanism: instead of a cookie, Umami computes a one-way **daily-rotating anonymous hash** from a few technical signals (IP address, browser type, and a salt that changes every midnight). That hash *is* the "visitor." The inputs are discarded; only the hash is stored. Nothing is planted on the patient's device — nothing to accept, decline, or clear.

Worked example: the same person, same machine, visiting repeatedly

WHEN THEY VISIT	WHAT THE DASHBOARD COUNTS	WHY
Monday 9am, 1pm, 6pm	**1 visitor, 3 visits**	Same day → same hash; a new "visit" (session) starts after ~30 minutes of inactivity
Same machine, Tuesday	**a new visitor**	The salt rotated at midnight → different hash; yesterday's identity is unrecoverable
Same person, phone then laptop	2 visitors	Different devices produce different hashes

Consequences to read the dashboard with: visitor counts are **slightly inflated** (returning people re-count daily), and **Retention/Cohorts are structurally understated** — the system is deliberately incapable of recognizing a returning person across days. Your own visits count like anyone else's; if that starts to skew small numbers, excluding practice traffic is a small future change.

Can we know *who* visited? No — and this is the load-bearing "no" of the whole design. No names, accounts, or persistent identifiers exist; analytics rows contain no IP addresses; the proxy access log truncates IPs and expires after 30 days. You can see aggregate character ("12 visitors from Austin on mobile, most arrived via Google") but not identity ("Jane Smith read the implants page") — not through the dashboard, the database, or full admin access. For a dental practice this inability is a **feature**: a list of identified people browsing dental services would itself be sensitive, health-adjacent information — exactly the category the HHS tracking-technology guidance warns about, and exactly why ad-tech analytics were rejected. Because the system cannot answer "who," the practice never inherits the obligations that answer would carry.

Boundary for the future: any suggestion to "identify users," "match visitors to the patient list," or connect analytics to form submissions is not a feature request — it is a re-architecture of the privacy posture, and it requires privacy/legal review *before* any code. The current design's value is that "who visited?" is **provably unanswerable**.

Every menu item, what it's for, and its status. "Menu visible" never means "collection enabled."

MENU	WHAT IT MEANS / EXAMPLE QUESTION IT ANSWERS	STATUS & HEALTHCARE NOTE
Overview	The headline numbers: visitors, visits, views, bounce rate, visit duration, top pages, referrers, devices. *"Was this week busier than last?"*	Enabled. Safe.
Events	Your five conversion events. *"How many people tapped Call this week?"*	Enabled — the most business-relevant screen. Safe (allow-listed data only).
Sessions	Anonymous visit-level list: pages in a visit, device, city-level location. *"What does a typical visit look like?"*	Enabled. Safe: identities are daily-rotating hashes; no cross-day tracking.
Realtime	Who's on the site right now (anonymously). *"Did the Facebook post drive traffic just now?"*	Enabled. Safe.
Performance	Page load timing as experienced by real visitors. *"Is the Invisalign page slow on phones?"*	Enabled (Umami web-vitals). Complements the planned Lighthouse CI.
Compare / Breakdown	Side-by-side periods; slice any metric by page, referrer, device, country. *"Mobile vs desktop conversion?"*	Enabled. Safe.
Goals	Set a target ("50 appointment clicks/month") and track progress.	Available; none configured yet. Safe — worth setting up once baselines exist.
Funnels	Step-by-step drop-off: landing → service page → appointment click. *"Where do we lose people?"*	Available; configure after 2+ weeks of data. Safe.
Journeys	Common page-to-page paths. *"Do blog readers reach service pages?"*	Enabled. Safe.
Retention	Return-visitor analysis. **Limited by design:** cookieless daily hashes mean cross-day retention is structurally understated. Read as trend, not truth.	Enabled but interpret with care.
Replays	Would record patients' actual scrolling/clicking sessions for playback.	NOT COLLECTED — MUST STAY OFF. The site loads no replay module VERIFIED (code). Recording patient browsing on a healthcare site requires privacy/legal review that has not been done. The menu existing is not consent.
Heatmaps	Click/scroll density pictures.	NOT COLLECTED — same rule as Replays: separate privacy review before ever considering.
Segments / Cohorts	Saved audience filters and grouped-by-first-visit analysis.	Available; more useful with volume. Cohorts share Retention's cookieless limitation.

Bounce rate

Share of visits that saw one page and left. High bounce isn't automatically bad — someone who lands on your phone number and calls "bounced" successfully.

Visit duration

Time between first and last action in a session. One-page visits contribute ~0s, dragging the average down — read alongside views-per-visit.

Referrers

Where visitors came from (Google, Facebook, direct). Direct = typed/bookmarked/most messaging apps.

Conversion rate

Defined here as sessions containing ≥1 approved conversion event ÷ total sessions (session-based, matching Umami's model).

Why booking completion is invisible

The moment a patient clicks "Book Online," they leave your website for Zocdoc. Tracking deliberately stops at that handoff — following patients into the booking system would cross the privacy line this design enforces. So `appointment_click` means "handed to Zocdoc," not "appointment confirmed." Confirmed-booking counts live in your Zocdoc account; comparing the two monthly gives you the handoff-to-booking rate.

9. Where is my analytics data stored?

> **The analogy:** Umami is the **librarian** — it receives each note, files it, and answers your questions. PostgreSQL is the **catalog and record system** — the actual organized records. The EBS volume is the **shelf** the records physically sit on. The backup is a **protected copy in a separate vault** (S3), refreshed nightly, in case anything happens to the library.

Database facts

QUESTION	ANSWER	STATUS
Technology	PostgreSQL 16 (official container image)	VERIFIED
Where it runs	Docker container on the dedicated analytics EC2 instance, AWS region us-east-2 (Ohio)	VERIFIED
Logical database	`umami`	VERIFIED
Physical storage	`/data/pg` on a dedicated 20 GB gp3 EBS volume	VERIFIED
Encryption at rest	EBS volume encrypted (AWS-managed keys); backups SSE-KMS in S3; snapshots inherit volume encryption	VERIFIED
Encryption in transit	Browser→server: TLS. Umami→PostgreSQL: unencrypted, but the connection never leaves the single host's private container network (port 5432 is not network-exposed) — accepted for the pilot	VERIFIED (by design)
Publicly accessible?	No. Only ports 443/80 are open, and only Caddy listens there. The database accepts connections solely from the Umami container on the same machine	VERIFIED
Credentials	Generated randomly, stored as encrypted SecureStrings in SSM Parameter Store, read once at boot; never in code, images, or Terraform state outputs	VERIFIED
What's stored	Pageview rows (path, referrer, device/browser class, country/region), the five approved event types with enum-only properties, daily-rotating anonymous session hashes	VERIFIED
Deliberately NOT stored	Names, emails, phone numbers, form contents, free text, cookies, cross-day identifiers, query strings, fragments, IP addresses in analytics rows (proxy access log truncates IPs; 30-day retention)	VERIFIED (code + design)
Retention	13 months (deliberate policy; Umami keeps data forever unless told otherwise — this was explicitly configured as a control)	CONFIGURED — verify the deletion job after month 13
Who may access	Umami app (service account); humans only via AWS-authenticated SSM sessions	VERIFIED

	(logged)	
Access audit	AWS CloudTrail (API calls) + SSM session logging; no direct DB user accounts exist to audit	**INFERENCE (standard AWS behavior; session-log review not yet performed)**

10. Security & privacy

> **The sentence that governs everything:** Self-hosted, privacy-first, and HIPAA-ready are **not interchangeable terms**. Compliance depends on the complete configuration, a documented risk analysis, operational safeguards, agreements, and what data is actually collected — not on any single technology choice. This system implements strong technical minimization; the practice's documented risk analysis remains the compliance artifact. **Legal/privacy review of the overall program is the owner's action item.**

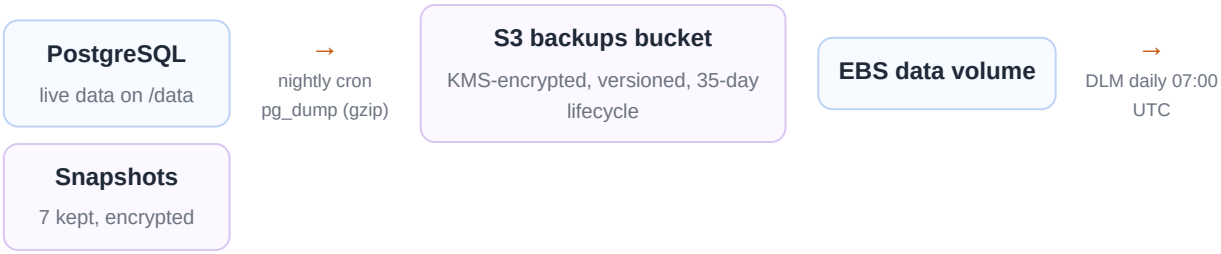
Control-by-control status

CONTROL	IMPLEMENTATION	STATUS
Cookieless	Umami tracker sets no cookies/localStorage identifiers; verified in staging DevTools before production approval	**VERIFIED** (re-check after upgrades)
Replay / heatmaps	No collection modules loaded by the site; policy: never enable without separate review	**VERIFIED (code)**
Form values / free text / identifiers	Structurally excluded: enum-only properties; email/phone/UUID/long-text patterns rejected; rejected values never echoed (tested)	**VERIFIED — 125 tests**
Query strings / fragments	Excluded at the tracker (`data-exclude-*`) AND at the wrapper (path normalization + allow-list)	**VERIFIED (two layers)**
HTTPS everywhere	CloudFront (site) + Caddy/Let's Encrypt (analytics); HTTP only for cert issuance/redirect	**VERIFIED**
Authentication	Dashboard: Caddy basic-auth + Umami login (strong password — the primary boundary since /api/* bypasses the Caddy gate by SPA necessity). Admin: SSM only, no SSH	**VERIFIED**
Least privilege	Instance role: read own 3 secrets, write backups, push logs — nothing else. CI plan role read-only. No wildcards on resources	**VERIFIED**
Network restriction	Security group: inbound 443+80 only; DB port unexposed; IMDSv2 enforced	**VERIFIED**
Patching	OS: dnf-automatic security updates. Containers: monthly review — **currently unpinned images**	**PARTIAL — see §15**
Secrets	SSM SecureStrings; placeholder-then-set pattern; one gap during rollout (placeholder boot) was caught and corrected same-day	**VERIFIED**
Known accepted risks	(a) /api/auth/login is internet-reachable behind Umami auth only — strong password mitigates; rate-limit recommended. (b) Basic-auth password transited a work chat — rotation recommended post-pilot	**See §15**

11. Backups & recovery

Backup & recovery flow

Two independent paths protect the data; restore rebuilds the machine from code and reloads the records.

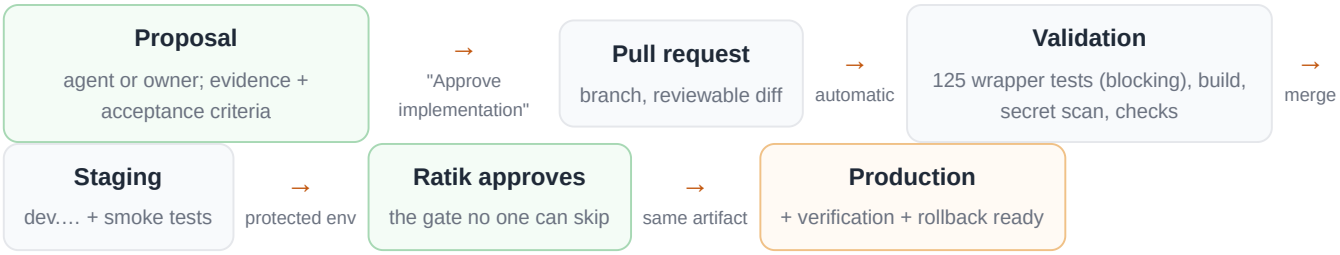


Restore path (RTO ≤ 4h, RPO ≤ 24h): re-apply Terraform (new instance boots from corrected bootstrap) → restore latest dump into fresh PostgreSQL → verify dashboard + ingest. Alternative: attach a snapshot-restored volume. **Honest status:** the stack is <1 day old — the `first nightly dump is unverified` and the `restore drill has not been performed`. Both are required before this section can claim "proven." What's lost in a total failure between backups: up to 24h of analytics rows — never website function, never patient data (there is none).

12. Deployment & change management

Deployment & approval flow (website changes, incl. the tracker)

One artifact is built once and promoted unchanged; a human gate sits before production, always.

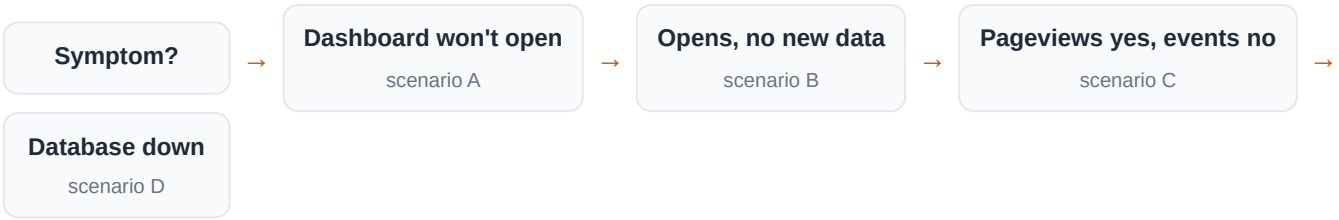


Infrastructure changes follow the same shape with Terraform: PR → automatic read-only *plan* → owner reviews the plan → explicit "Approve Terraform apply" → apply (this stack: via CloudShell admin, because the pipeline's apply role deliberately lacks the permissions — documented, not accidental). The agent can propose and prepare; it cannot merge, approve, or apply.

13. Troubleshooting

Decision flow — start with the symptom

Each branch below expands in the scenarios that follow.



A — Dashboard does not open

1. **Is it just you?** Try a fresh incognito window first (cached basic-auth causes most "hangs" — it did during rollout).
2. **DNS:** does `analytics.nextgendentalaustintx.com` resolve? (Route 53 A-record → instance's static IP.)
3. **Certificate:** browser padlock error → Caddy's Let's Encrypt renewal failed; check Caddy container logs via SSM.
4. **Instance:** EC2 console → status checks (the CloudWatch alarm should have emailed you already).
5. **Containers:** SSM session → `sudo docker ps` — all three Up? A restart-loop names the guilty container; `sudo docker logs <name>` tells you why (during rollout, a Caddyfile syntax error looked exactly like this).
6. **Auth:** basic-auth rejects → verify against the stored hash flow in the runbook; Umami login rejects → password manager, then reset via SSM if needed.

B — Dashboard opens but shows no new data

1. **Date filter first** — "Last 24 hours" vs "This week" catches many false alarms.
2. **Your own ad-blocker:** many block `/script.js` or `/api/send`; test in a clean profile before suspecting the system. (Some patient browsers will always block — accept the undercount.)
3. **Tracker loading?** On the live site: DevTools → Network → is `script.js` fetched (200)? Is `/api/send` POSTing on navigation?
4. **Right website ID?** The payload's `website` field must match the site you're viewing in Umami (dev and production are separate).
5. **Server side:** SSM → Umami container logs for ingest errors; then scenario D checks.

C — Pageviews work but conversion events don't

1. Browser console on the live site: the wrapper logs rejection *codes* (never values) — `[analytics] rejected: …` names the exact check that failed.

2. Common causes in order: event fired on an untracked host; a property value not on the enum allow-list; a required property missing; the route not on the path allow-list (aggregated to `/category/…` — check Events for those); unsafe-path rejection (see the aggregate counter).
3. If a NEW page type was added to the site, the wrapper's route allow-list may need extending — that's a normal PR, and exactly what the "unrecognized route" warning counter exists to surface.

D — Database is unavailable

How it appears: dashboard errors or empty panels; Umami logs show connection failures; the site itself is completely unaffected. **Events during the outage are lost, not queued** — the tracker fails silently by design (an analytics outage must never break the website). **Checks:** SSM → `sudo docker ps` (is `db` Up?), `sudo docker logs umami-db-1`, disk space on `/data` (the 80% alarm should precede this). **Recovery ladder:** restart the container → reboot the instance → restore last night's dump to a fresh instance (§11) — that last step is the only one that loses data (≤24h of analytics rows).

E — "The dashboard doesn't match Google"

It never will, and that's expected: **Google Business Profile** counts interactions with your Maps/Search listing (calls from the listing, direction requests) — most of those people never touch your website. **Search Console** counts search impressions and clicks — before arrival. **Umami** counts behavior on the site itself — after arrival. Each tool also defines visitors, sessions, and bots differently, and ~10–25% of browsers block trackers (Umami undercounts those; Google's listing metrics are unaffected). Use each for its layer: GBP = listing, GSC = search door, Umami = inside the building.

14. Costs & ownership

Monthly running cost (estimates verified at provisioning; confirm on the first full AWS bill)

ITEM	EST. $/MONTH	NOTES
EC2 t4g.small (analytics host)	~12.30	On-demand; the dominant cost
EBS storage (8 GB root + 20 GB data)	~2.25	gp3, encrypted
Snapshots + S3 backups	~0.60	Incremental snapshots; small dumps
CloudWatch (alarm + 30-day logs)	~1.20	
Route 53 record, SNS email, SSM, TLS	~0.00	Let's Encrypt is free; ACM is free
Total analytics stack	**~$16–17**	Plus ~1–2 h/month operator time (patch review, backup check; quarterly restore drill)

Ownership: Ratik owns and approves everything — AWS account, repository, dashboard admin, production gate. The agent (this system's AI assistant) proposes, implements to branches, monitors, and documents — with no deploy, merge, or approval power. Clinical content claims route to Dr. Kondragunta / Dr. Yanala for review. There is no third-party analytics vendor relationship to manage — that's the point of self-hosting; the trade is that the ~1–2 h/month of operational ownership is yours.

15. Risks & recommendations

Prioritized; nothing here was changed by this documentation task

PRIORITY	ITEM	WHY / RECOMMENDED ACTION
CRITICAL	Restore drill never performed	A backup that has never been restored is a hope, not a backup. Run the §11 drill this week; record the result. (Also verifies the first nightly dump.)
HIGH	Container images unpinned (`postgresql-latest`, `postgres:16`, `caddy:2`)	An upstream release could change behavior (including re-enabling features like replay) on any instance replacement. Pin digests in the bootstrap; bump deliberately, re-run the privacy spot-check after each bump.
HIGH	Infrastructure code not merged (`infra/analytics-pilot` PR open)	Live infra ≠ main branch until merged. Merge after the plan legs are green so the code that built production lives on main. Includes the dev-stack "1 add / 1 change" diff — identify before merging.
MEDIUM	Single point of failure (one instance)	Accepted for the pilot (outage loses analytics rows only, never website function). Revisit managed RDS/Fargate (~$42–48/mo) only if the pilot graduates and ops time grows.
MEDIUM	Login endpoint internet-reachable behind Umami auth only	Strong password mitigates; add a Caddy rate-limit on `/api/auth/login` in the next bootstrap revision.
MEDIUM	Basic-auth password transited a work chat during rollout	Rotate it (5-minute runbook step) now that the pilot is proven.
MEDIUM	Program-level privacy documentation	Technical minimization is strong; the practice's documented risk analysis / legal review of the analytics program is the remaining compliance artifact. Owner + counsel.
LOW	Retention deletion job unproven (fires at month 13)	Calendar a check in ~12 months; watchdog can carry the reminder.
LOW	Alarm never test-fired	Schedule a controlled stop/start drill to see the email arrive end-to-end.

Current strengths worth naming: privacy enforced in code with blocking tests, not policy documents; two independent auth layers with no SSH surface; encrypted everything; two backup paths; complete change history in Git/Actions; the website is structurally immune to analytics failures; and every production change passes a human gate.

16. Glossary

Astro

The framework that builds the website into plain static files — no server code runs when patients browse.

CDN / CloudFront

A network of servers near users that serves your site fast and shields the origin.

S3 / EBS / snapshot

AWS storage: S3 = durable object storage (files/backups); EBS = a virtual disk attached to a server; snapshot = a point-in-time copy of that disk.

EC2 / instance

A rented virtual server. Yours is a small ARM machine ("t4g.small").

Docker / container

A way to run applications in isolated boxes with their dependencies. Your three: Caddy, Umami, PostgreSQL.

Reverse proxy (Caddy)

The front-door program that receives all traffic, handles HTTPS, and routes/guards requests.

PostgreSQL

The database — organized, queryable storage for every analytics record.

Terraform / IaC

Infrastructure written as reviewable code, so the whole stack can be recreated or destroyed deliberately.

CI/CD pipeline

The automated assembly line: test → build → staging → (human gate) → production.

OIDC role

How the pipeline gets short-lived AWS permissions without stored passwords.

SSM

AWS Systems Manager — encrypted parameter storage and the logged, SSH-free admin console.

TLS / HTTPS

The encryption on every connection (the padlock).

Session / visitor hash

Umami's anonymous, daily-rotating stand-in for "a person" — the reason nobody can be tracked across days.

RPO / RTO

How much data you can lose (RPO ≤ 24h here) and how long recovery takes (RTO ≤ 4h target).

PHI

Protected Health Information — what this system is designed never to touch.

17. Evidence sources

- Repository `ratikk/NextGen-Dental` at main (`fa70d23`): `src/components/Analytics.astro`, `src/utils/analytics/*` (wrapper, 125-test suite, types), `src/utils/forms.ts`, `src/layouts/Layout.astro`, workflows (`pr-validation.yml`, `deploy.yml`, `rollback.yml`, `terraform.yml`).
- Branch `infra/analytics-pilot` (`98e3610`): `infra/analytics/` Terraform (~25 resources), `user-data.sh` bootstrap, `docs/ANALYTICS.md`.
- Terraform plan and apply outputs from the approved 2026-08-09 CloudShell session (25 added / 0 changed / 0 destroyed; replacement apply 3/2/3).
- Live instance evidence via SSM sessions: `docker ps`, Caddy logs (certificate obtained), cloud-init log, live Caddyfile.
- Live endpoint checks: `script.js` serving (200); dashboard reachable behind both auth layers.
- Owner-performed staging verification (DevTools payload/cookie checks) preceding production approval, 2026-08-09; owner's dashboard screenshots showing dev + production collecting.
- Project records: analytics release record, outcome log, CETO charters, SEO audit.

18. Open questions

1. Did the first nightly `pg_dump` land in the backup bucket? (Check the morning after go-live; then weekly via watchdog.)
2. Restore drill date? (Critical item — §15.)
3. Which Umami application version is actually running? (Surfaces when pinning digests.)
4. Was `chmod 600` applied to the two config files on the instance? (One SSM command to confirm.)
5. What is the dev-stack plan's "1 to add, 1 to change"? (Unrelated stack; identify before merging the open infra PR.)
6. Per-item record of the staging verification checklist (production was approved; item-level evidence should be written into the release record for the audit trail).
7. Date for the practice's documented privacy risk analysis covering the analytics program?
